

Umoya Project Compilation

Hackathon Planning Documents

September 2026

Part 1: Project Strategy & Technical Plan

1. The project in one sentence
2. The problem statement
3. The solution
4. The killer idea: shares
5. What makes this different from Lido/Aave?
6. MVP scope
7. Your UI philosophy is excellent
8. Recommended tech stack
9. Smart-contract architecture
10. Risk architecture
11. Risk 2 — External protocol risk
12. Risk 3 — Asset/liquidity risk
13. Risk 4 — Social/group risk
14. Risk 5 — Admin/custody risk
15. Risk 6 — Oracle / valuation risk
16. One thing I would change from the original proposal
17. Early withdrawal penalty
18. Your developer notes
19. Three-developer allocation
20. Your 72-hour execution plan
21. What I would put in the submission
22. Your architecture slide
23. The biggest thing I'd change from your original plan

Part 2: Project Description — Background & Overview

What I would change

Background: What inspired / motivated this project?

Overview: What is the project about?

Core value proposition

One more important distinction

Part 1: Project Strategy & Technical Plan

Yes. Taking everything you pasted together, I would **reframe the project away from “a DeFi staking dashboard” and toward “a programmable on-chain stokvel.”**

That gives you a much clearer problem, solution, technical story, and risk story.

One important update from the material you pasted: **don't build the hackathon plan around Sepolia + Lido.** Lido's current documentation says its Sepolia deployment is deprecated and recommends Hoodi instead; Aave V3 does have a Base Sepolia deployment. For a 72-hour project, I would therefore separate the **demo/test architecture** from the **intended production architecture** rather than forcing an outdated Sepolia integration. ([Lido Docs](#))

1. The project in one sentence

A decentralized stokvel that allows groups of people to pool ETH, automatically deploy their collective capital into a yield-generating strategy, and distribute the resulting returns transparently according to each member's contribution.

The important word is **group**.

You aren't trying to compete with Lido.

You aren't trying to compete with Aave.

You're building the **social coordination layer on top of existing DeFi infrastructure**.

2. The problem statement

I'd use something close to this for your submission:

Problem

Traditional stokvels and community savings groups enable people to build wealth collectively, but their financial management is often dependent on manual contributions, spreadsheets, bank accounts, and a trusted organizer.

At the same time, decentralized finance gives individuals access to staking and yield-generating opportunities, but these products can be difficult for ordinary users to understand and operate. Users are expected to navigate wallets, protocols, tokens, smart contracts, and unfamiliar financial terminology.

This creates a gap between **community-based saving** and **on-chain financial infrastructure**.

People should not need to understand liquid staking, lending markets, token wrappers, or DeFi protocols simply to participate in a collective savings strategy.

Our project addresses this gap by creating a decentralized group savings pool that abstracts the underlying DeFi infrastructure.

Members contribute ETH to a shared smart contract. The protocol manages the group's capital through an approved yield strategy and tracks each member's proportional ownership. When the underlying strategy generates returns, those returns are reflected in each member's share of the pool.

The result is a familiar stokvel experience for the user, while blockchain provides transparent accounting, programmable rules, and verifiable ownership underneath.

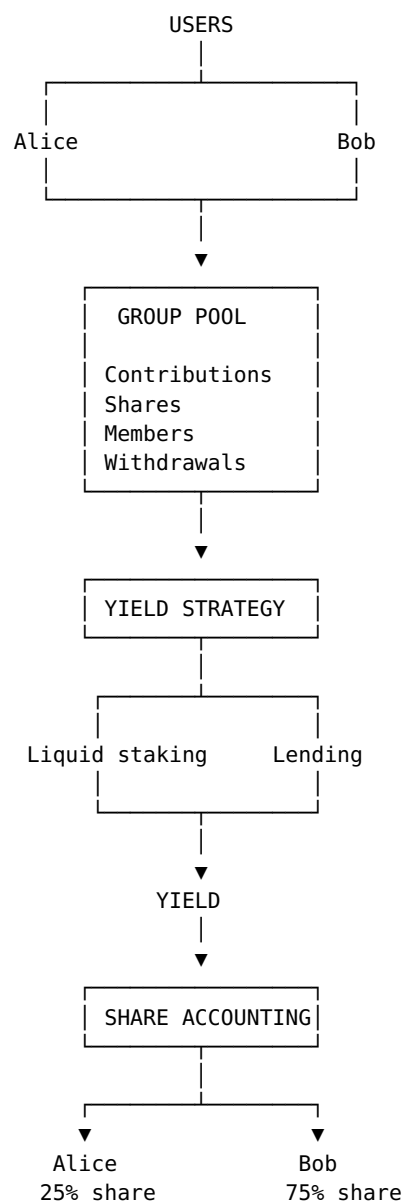
That's much stronger than:

"We are building a platform that stakes ETH through Lido and lends it through Aave."

Because **Lido and Aave are implementation details**.

3. The solution

Your architecture becomes:



The user doesn't need to know whether the strategy underneath uses:

- Lido
- Aave
- another lending protocol
- another staking protocol

That becomes your **strategy layer**.

4. The killer idea: shares

This is the technical concept I would emphasize to judges.

Suppose:

Alice deposits 1 ETH

Bob deposits 3 ETH

Pool = 4 ETH

Alice = 25%

Bob = 75%

The pool subsequently generates:

+0.4 ETH

The pool is now:

4.4 ETH

Alice owns:

$25\% \times 4.4 = 1.1$ ETH

Bob owns:

$75\% \times 4.4 = 3.3$ ETH

Nobody needs to manually calculate everyone's profit.

The contract's accounting determines ownership.

That's a very good smart-contract demonstration because it shows **why blockchain is useful here**, rather than merely putting a normal savings application on a blockchain.

5. What makes this different from Lido/Aave?

This question is almost guaranteed to come up.

Your answer:

Lido and Aave provide financial infrastructure. We provide the group coordination and ownership layer.

Think of it as:

Lido/Aave

↓

Financial infrastructure

Your protocol

↓

Social + ownership infrastructure

The innovation isn't necessarily the underlying yield mechanism.

It's:

collective participation + programmable ownership + automated distribution.

6. MVP scope

For 72 hours, I would be ruthless.

MUST HAVE

Connect wallet

↓

Create group

↓

Join group

↓

Deposit ETH

↓

View contribution

↓
View pool balance
↓
View ownership %
↓
Withdraw

And if your DeFi integration works:

Pool
↓
Yield strategy
↓
Yield
↓
Updated member balances

That's your MVP.

CUT FROM THE MVP

Don't waste hackathon hours on:

- chat
- DAO governance
- complicated charts
- AI
- custom validator infrastructure
- cross-chain functionality
- leverage
- borrowing
- liquidation mechanisms
- complicated NFTs
- token launches
- elaborate analytics

You can put those under:

Future Development

7. Your UI philosophy is excellent

I'd keep the minimalist UI.

Home

Your Savings

2.40 ETH
+0.08 ETH earned

Friends Pool
12.4 ETH
+4.2%
8 people

Family Pool
3.1 ETH
+3.8%
4 people

+ Create Pool

Pool

Family Pool

3.10 ETH

+0.12 ETH earned

4 people

Your share

0.80 ETH
25%

[Add ETH]

[Withdraw]

That's enough.

The underlying complexity should be invisible.

8. Recommended tech stack

For **three developers / 72 hours**, I'd use:

Layer	Technology
Smart contracts	Solidity
Contract development	Foundry
Security libraries	OpenZeppelin
Frontend	Next.js + TypeScript
Ethereum interaction	Wagmi + Viem
Wallet	RainbowKit or native Wagmi
Styling	Tailwind CSS
Blockchain	Base / Base Sepolia
DeFi integration	Aave V3 / compatible strategy
Indexing	Events + frontend reads initially
Testing	Foundry
Deployment	Foundry scripts
Git	GitHub
RPC	Alchemy / QuickNode / Infura
Contract verification	Block explorer

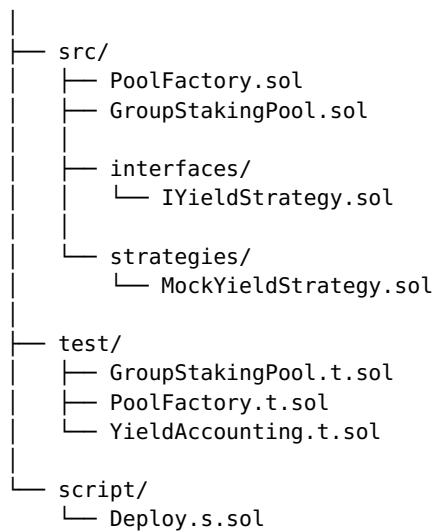
Aave officially lists both **Base V3 and Base Sepolia V3**, which makes Base a reasonable target for this architecture. (aave.com)

And wstETH is specifically designed as a wrapper that integrates liquid staking with other DeFi protocols, including L2 environments. ([Lido Docs](#))

9. Smart-contract architecture

I'd actually simplify the original folder structure.

contracts/



Why IYieldStrategy?

This is an excellent developer decision.

Your pool shouldn't care whether the strategy is:

```

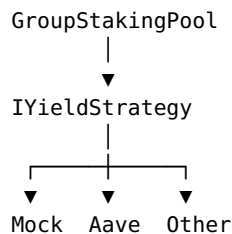
Lido
Aave
Spark
Mock strategy
  
```

It should simply say:

```

deposit assets
withdraw assets
get current value
  
```

That gives you:



During development you can use a mock.

Then plug in the real protocol.

This dramatically reduces hackathon risk.

10. Risk architecture

This is where your project can look mature.

Don't pretend the system is risk-free.

Instead, explicitly identify the risks.

Risk 1 — Your smart contract

Your contract controls pooled funds.

Potential problems:

- incorrect accounting
- reentrancy
- unauthorized withdrawals
- incorrect share calculations
- rounding errors
- faulty strategy interactions

Mitigation

Use:

- OpenZeppelin ReentrancyGuard
- checks-effects-interactions
- carefully tested accounting
- minimal privileged functions
- Foundry unit tests
- invariant/fuzz testing where possible
- no unnecessary complexity

11. Risk 2 — External protocol risk

If your pool interacts with another DeFi protocol, **your protocol inherits some of its risks.**

For example:

Your Pool
↓
Aave
↓
Underlying assets

If the underlying protocol suffers an exploit or operational issue, your users can be affected.

Therefore don't say:

| "Our funds are completely safe because Aave/Lido is audited."

Instead:

| "We reduce implementation risk by integrating established protocols, but external protocol risk remains."

That's a much more credible answer.

12. Risk 3 — Asset/liquidity risk

This is particularly important.

If:

ETH
↓
Liquid staking token
↓
Lending protocol

the user's claim isn't necessarily equivalent to:

instant ETH withdrawal at exactly 1:1

There can be:

- liquidity constraints
- price movements
- withdrawal delays
- market discounts
- protocol-specific withdrawal mechanics

Lido's documentation explains that wstETH represents shares of stETH and that its value changes relative to stETH rather than its token balance rebasing. ([Lido Docs](#))

So your UI shouldn't promise:

| "Deposit 1 ETH → guaranteed 1.05 ETH."

Instead show:

Your current share
0.84 ETH

Current pool value
3.36 ETH

and make the actual strategy risks clear in your documentation.

13. Risk 4 — Social/group risk

This is actually your **unique risk category**.

Imagine:

Alice → 9 ETH
Bob → 0.1 ETH
Carol → 0.1 ETH

Alice owns ~98%.

That's not necessarily a bug.

It's simply the consequence of proportional ownership.

You therefore need clearly defined rules for:

- who can join
- whether the pool has a maximum size
- whether deposits are allowed after the group begins
- when withdrawals are allowed
- how early withdrawals work
- whether there are penalties
- who can trigger strategy operations

These aren't just UI decisions.

They're **protocol rules**.

14. Risk 5 — Admin/custody risk

Avoid designing:

Admin
↓
can withdraw everyone's ETH

That completely undermines your decentralization story.

Instead:

User
↓
Pool contract
↓
Strategy

The application frontend should not have custody of user funds.

If you need an emergency/admin mechanism, make its powers extremely narrow and document exactly what it can and cannot do.

15. Risk 6 — Oracle / valuation risk

If you display:

Your pool = \$4,800

you need some mechanism for determining the ETH/token value.

For the hackathon, you don't need to over-engineer this.

But if your protocol eventually relies on price feeds for withdrawals, limits, or accounting, use established oracle infrastructure rather than trusting a frontend-provided price.

Lido's documentation, for example, documents Chainlink-compatible wstETH/stETH rate feeds on Base and other networks. ([Lido Docs](#))

16. One thing I would change from the original proposal

I would **not promise time-weighted yield in the MVP**.

The original idea:

contribution × time

sounds clever, but it introduces significant accounting complexity.

You already have:

deposits
shares
yield
withdrawals
strategy accounting

Adding:

time-weighting

creates another class of bugs.

For the hackathon:

Version 1

Member ownership =
member shares / total shares

Simple.

Transparent.

Easy to test.

Then:

Version 2

time-weighted participation

can become a future feature.

17. Early withdrawal penalty

This is another feature I'd be careful with.

Instead of immediately implementing:

2% penalty

first implement:

withdraw()

correctly.

Then, if everything works, add:

early withdrawal

↓

small protocol-defined fee

↓

remaining pool

The important thing is that the rules are transparent and enforced by the contract.

Don't introduce economic incentives that you haven't thoroughly tested just because they sound good in a pitch.

18. Your developer notes

These are the things I'd tell your three developers before they start.

Developer Note #1

Separate accounting from strategy.

Your pool should know:

who deposited

how many shares they own

how much the pool is worth

It shouldn't contain every detail of Aave/Lido.

Developer Note #2

Never trust the frontend for financial calculations.

Frontend:

display

Contract:

truth

If the frontend says:

You own 25%

the contract should independently determine that value.

Developer Note #3

Use events heavily.

For example:

GroupCreated
MemberJoined
Deposited
Withdrawn
StrategyUpdated

This makes debugging and later indexing dramatically easier.

Developer Note #4

Test the accounting before integrating DeFi.

Your first tests should prove:

Alice deposits
→ shares increase

Bob deposits
→ shares increase

Yield increases pool value
→ ownership percentages remain correct

Alice withdraws
→ her proportional value leaves the pool

Bob's remaining ownership remains correct

If these aren't bulletproof, don't touch Aave.

Developer Note #5

Use a mock strategy.

This is probably the biggest hackathon productivity trick.

Pretend your strategy generates:

+10%

Then test the whole application.

Once everything works:

MockStrategy
↓
RealStrategy

You're no longer debugging your UI, pool accounting and Aave integration simultaneously.

19. Three-developer allocation

Developer 1 — Protocol

PoolFactory
GroupStakingPool
shares
deposits
withdrawals
events
tests

Developer 2 — Frontend

Next.js
Wagmi
Viem
wallet connection
Home
Create Pool
Pool page
transactions

Developer 3 — DeFi + integration

Strategy interface
Mock strategy
Aave integration
deployment scripts
contract verification
README
demo environment

And **everyone tests together**.

20. Your 72-hour execution plan

Hours 0-12

Get this working:

Wallet
↓
Create pool
↓
Join pool
↓
Deposit ETH

No DeFi.

Hours 12-24

Implement:

shares
ownership %
withdrawal
events
tests

At this point you should already have a functional blockchain application.

Hours 24-40

Build:

MockYieldStrategy

↓

Pool

↓

Yield

↓

Updated balances

Now your entire product works without external protocol dependencies.

Hours 40-55

Integrate the actual DeFi strategy.

This is where you determine whether the intended deployment environment and protocol contracts are actually usable.

Don't discover this during the final three hours.

Aave's current documentation confirms Base and Base Sepolia V3 deployments, while Lido's current docs show that its old Sepolia deployment is deprecated, so verify the exact testnet strategy before locking the demo architecture. (aave.com)

Hours 55-65

Polish:

UI

animations

mobile

error handling

transaction states

loading states

empty states

Hours 65-72

STOP CODING FEATURES.

Use the remaining time for:

testing

demo rehearsal

README

architecture diagram

pitch

backup demo

video

21. What I would put in the submission

Project name

Something like:

ETH Chama

or

ChamaFi

or

StokETH

The name should communicate the idea immediately.

Tagline

| **A decentralized stokvel for collective ETH growth.**

Or:

| **Save together. Grow together. Own it on-chain.**

Problem

| Community savings groups are powerful, but their administration is often manual and their members have limited access to programmable financial infrastructure. Meanwhile, DeFi yield opportunities are accessible primarily to users who understand complex wallets, protocols and financial products.

Solution

| We turn a traditional stokvel into a programmable on-chain group savings pool. Members contribute ETH to a smart contract, receive proportional ownership, and collectively participate in a yield strategy without needing to interact directly with the underlying DeFi protocols.

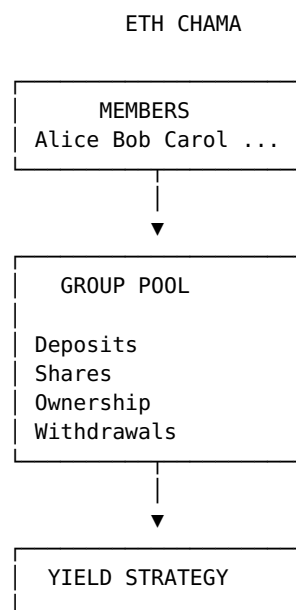
Innovation

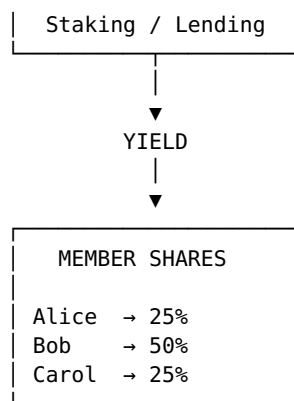
| **We don't reinvent DeFi. We make DeFi communal.**

That is probably your strongest sentence.

22. Your architecture slide

Put this on one slide:





23. The biggest thing I'd change from your original plan

Don't pitch:

“We give African users access to Lido + Aave.”

That makes you sound like another DeFi frontend.

Pitch:

“We bring the stokvel model on-chain and use DeFi as the financial engine underneath it.”

Then your technology becomes:

Stokvel
+
Smart contracts
+
Collective ownership
+
DeFi yield
=
ETH Chama

And that gives the judges a very simple story:

People already know how a stokvel works.

You're simply asking:

What happens if the rules of that stokvel live on Ethereum instead of inside someone's spreadsheet?

That's the conceptual core I'd build the entire hackathon submission around.

Part 2: Project Description — Background & Overview

Yes — **the description is aligned with the project concept**, but I would make a few important changes before submitting it.

The biggest issue is that the **Background is strong**, while the **Overview currently promises several things that may be too ambitious or technically inaccurate for a 72-hour hackathon**.

In particular, I would not submit the current wording that says the protocol will “*automatically liquid-stake via Lido to mint stETH, then immediately supply that stETH into Aave V3*” unless your team has actually verified the exact deployment you will use. Lido’s Sepolia deployment is now deprecated and Lido recommends Hoodi for testnet integrations. Aave V3 is available on Base and Base Sepolia. ([Lido Docs](#))

Also, for current Aave integrations, **wstETH rather than the old direct stETH Aave V2 integration** is the relevant Lido/Aave integration pattern. ([Lido Docs](#))

What I would change

Your conceptual flow should be:

Community → Group Pool → Yield Strategy → Collective Returns

rather than making Lido + Aave the identity of the project.

That’s important because **Lido and Aave are infrastructure; Umoya is the product.**

I would submit this:

Background: What inspired / motivated this project?

Township economies rely heavily on stokvels and chamas as mechanisms for collective saving and financial resilience. However, traditional group savings models can face challenges around trust, transparency, administration, and access to more sophisticated financial opportunities.

In a conventional savings group, members often depend on a central organizer or institution to manage pooled funds. This creates a point of trust and operational dependency: members may have limited visibility into how funds are managed, while record-keeping, contributions, withdrawals, and distribution of returns can remain largely manual.

At the same time, decentralized finance provides access to programmable financial infrastructure, staking, and lending markets, but these systems are often technically complex for ordinary users. Individuals must navigate wallets, tokens, smart contracts, and multiple protocols before they can participate.

Umoya was inspired by the opportunity to bring these two worlds together.

We aim to preserve the familiar social structure of a stokvel while using blockchain to provide transparent ownership, programmable rules, and verifiable accounting. Instead of requiring every member to understand the underlying DeFi infrastructure, Umoya abstracts that complexity behind a simple group savings experience.

Overview: What is the project about?

Umoya is a decentralized group-staking and yield protocol inspired by the traditional stokvel model.

Members create or join a group and contribute ETH to a shared smart contract. The contract records each member's contribution and proportional ownership of the pool. The pooled capital can then be deployed through an approved yield-generating strategy, allowing the group to collectively participate in DeFi without each member having to interact directly with multiple underlying protocols.

The protocol's core innovation is the combination of **social group savings, programmable ownership, and DeFi yield generation**.

Users interact with a simple interface showing only the information they need: the group's total value, their personal share, accumulated returns, and the number of participants. The underlying DeFi infrastructure is abstracted away.

For the hackathon implementation, Umoya will demonstrate this architecture using an appropriate liquid-staking and/or lending strategy on a supported Ethereum-compatible network. Where supported, liquid-staked ETH can be represented by assets such as stETH/wstETH and supplied to compatible DeFi markets such as Aave V3. Aave currently supports V3 deployments on networks including Base and Base Sepolia, while Lido's current primary testnet is Hoodi rather than Sepolia. (aave.com)

Umoya therefore does not attempt to replace existing DeFi protocols. Instead, it creates a **social coordination and ownership layer on top of them**.

The protocol is designed around proportional ownership: a member's claim on the pool is determined by their share of the pool, allowing generated returns to accrue automatically according to ownership.

Future versions may introduce additional mechanisms such as time-weighted participation, early-withdrawal penalties, and other group-specific rules.

Core value proposition

Umoya makes DeFi collective.

Traditional stokvels provide the social structure for people to save together. DeFi provides programmable financial infrastructure. Umoya combines the two into a single experience where communities can pool capital, participate in yield-generating strategies, and transparently track their ownership on-chain.

The user experience remains simple:

Create a group → Contribute → Grow collectively → Withdraw your share.

Why I prefer this version

It keeps your original vision but avoids making claims you may not be able to demonstrate.

Your original says:

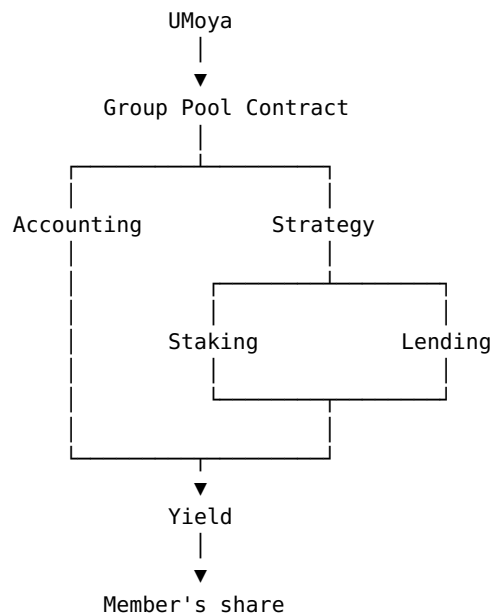
“automatically liquid-stakes the principal via Lido to mint stETH, then immediately supplies that stETH into Aave V3”

That's a **specific implementation**.

The stronger submission says:

“The pooled capital can be deployed through an approved yield-generating strategy.”

Then your architecture can show:



That gives you flexibility if the actual protocol integration changes during the hackathon.

One more important distinction

I would **not call the pool “trustless” without qualification.**

Your smart contract can remove the need to trust a traditional fund manager for the basic custody/accounting rules, but users still take risks from:

- your smart contract;
- external DeFi protocols;
- market/liquidity conditions;
- the strategy itself;
- blockchain infrastructure;
- governance/admin mechanisms, if any.

So your submission should say **“non-custodial and transparently governed by smart-contract rules”** rather than implying there is zero trust or risk.

And I would definitely avoid:

“institutional-grade DeFi yields”

That’s marketing language that invites a judge to ask, *“What exactly makes your yield institutional-grade?”*

Instead:

“access to established DeFi yield-generating infrastructure.”

Much easier to defend.

Your strongest positioning

If I had to reduce the whole project to one sentence for the judges:

Umoya brings the stokvel model on-chain, allowing communities to pool ETH, maintain transparent proportional ownership, and collectively access DeFi yield without requiring every member to understand the underlying protocols.

That is the **actual product**.

Lido/Aave are simply the machinery underneath it.